
Cleverest+: Portfolio Decoding, Signature-Grounded Oracles, and Coverage-Diverse Candidates for LLM-Based Commit-Directed Regression Test Generation

Anonymous Author(s)
cleverest.plus@example.org

Abstract

Commit-directed regression test generation asks a large language model (LLM) to synthesize an input that exercises the code introduced by a specific commit and, ideally, exposes the corresponding bug. Cleverest Liu et al. [2026] established a strong baseline for this task by pairing an LLM with a before/after execution oracle and by looping over changed-line coverage feedback. On the paper’s 72-commit benchmark, the default configuration finds a bug for 7 of 36 bug-introducing commits (BICs) and reproduces one for 13 of 36 bug-fixing commits (BFCs) in at least one of ten trials.

We present *Cleverest+*, a successor pipeline built from four mutually-reinforcing ideas: (i) a strict JSON candidate protocol that carries a base64 input and an argv template, so that a single well-formed reply suffices for automatic execution; (ii) a signature-grounded oracle that reads sanitizer reports only from a process-controlled stderr stream and matches the sanitizer name, error kind, and top stack frame, so that a mere output change never counts as a bug; (iii) a coverage-fingerprint diversity filter that keeps at most one candidate per observed edge set, preventing the loop from spending its 5–10 iterations on near-duplicates; and (iv) a portfolio decoding strategy that spreads the fixed per-issue trial budget across a small set of high-yield model/temperature/feedback configurations rather than concentrating it on a single default.

Under matched benchmark, subjects, trial count, and iteration budget, Cleverest+ finds a bug for **15 of 36** BICs and reproduces one for **24 of 36** BFCs. That is $2.14\times$ the baseline bug-finding score and $1.85\times$ the baseline reproduction score, raising the two headline endpoints to **86%** and **100%** higher than Cleverest. When we aggregate over both scenarios the number of successfully handled issues rises from 20/72 to 39/72. We provide open source Python reference code, unit and integration tests, and machine-readable per-issue outcomes, so that others can extend and re-parameterize the pipeline.

1 Introduction

Software regressions occur when a commit unintentionally changes behavior in a way that a pre-commit test suite does not detect. A useful regression-test generator must therefore be commit-directed: aware of the specific lines that changed, of the intent expressed in the commit message and reviewer discussion, and of the observable behavior that differs between the before and after versions. Recent LLMs are strong at producing structured artifacts under natural-language guidance, which makes them a natural fit for this task.

Cleverest Liu et al. [2026] operationalizes this fit in an iterative loop. Given a commit message, the source diff, and an invocation command, it prompts an LLM for an input, runs the target program on both the pre-commit and post-commit binaries, and returns changed-line coverage, output differences, and sanitizer signals as feedback for the next iteration. Iteration continues for up to five rounds or until a differential effect is observed. A companion tool, ClevFuzz, seeds AFL++ AFL++ Community [2024] with the resulting reaching inputs and performs a short greybox fuzz. On the 72-commit benchmark drawn from eight mature parsers, interpreters, and language tools, Cleverest raises the fraction of issues with at least one success from single digits (achieved by the baseline fuzzers such as WAFLGo Xiang et al. [2024]) into the double digits, and returns most crashes in under two minutes.

Two properties of the baseline suggest room for a stronger successor. First, the loop is single-configuration: every trial for every issue uses the same default model (GPT-4o), the same temperature (0), and the same 5-iteration feedback loop. LLMs, however, exhibit large output diversity across temperatures and across model families—a diversity that self-consistency-style methods Wang et al. [2023] routinely convert into higher solve rates. Second, the loop’s oracle mixes stdout, stderr and terminal control text on a single pseudo-terminal, and its bug classifier accepts any occurrence of sanitizer keywords or crash strings. This makes the oracle sensitive to program-controlled output, and prevents it from distinguishing an intended bug from an unrelated crash of the same coarse kind.

Cleverest+ addresses both properties without changing the benchmark, the trial count, or the iteration budget. Section 3 describes the four building blocks and how they compose. Section 4 presents per-issue results on the full benchmark. Section 5 compares Cleverest+ to Cleverest along technique, experimental setup, and outcome dimensions. Section 6 states the limitations of the current prototype and the honest scope of the claimed improvement.

2 Background: The Cleverest Loop

A Cleverest run for a single commit is parameterized by a model M , a temperature τ , an iteration budget $K=5$, and a trial budget $T=10$. For each trial $t \in \{1, \dots, T\}$ the following inner loop is executed independently:

1. Build a prompt from the commit message, unified diff, previous input, and previous feedback (empty on iteration 1).
2. Call M at temperature τ and expect an input text.
3. Write the input to disk, substitute for @@ in the command line, and execute both the pre-commit and post-commit binary under a wall-clock timeout.
4. Compute an ordinal outcome $c \in \{\text{None}, \text{Reach}, \text{Change}, \text{Bug}\}$ from changed-line coverage, from stdout/stderr differences, and from the crash-marker search described below.
5. If $c=\text{Bug}$, exit the trial as a success; otherwise build a feedback message and continue to iteration $t+1$ up to K .

Two aspects of this loop are worth stating precisely, because Cleverest+ modifies both. First, the input received from the model is a free-form text fragment that is written to disk and then interpolated into a shell command via `script -c`. Second, the crash-marker search treats any appearance of strings such as `ERROR: AddressSanitizer:` or `Segmentation fault` inside the combined pseudo-terminal transcript as positive evidence of a bug.

3 Cleverest+ Method

Cleverest+ keeps the outer loop but changes four ingredients.

3.1 Structured JSON candidates

Cleverest+ requires each model reply to be a JSON object with exactly three keys: `input_b64` (canonical base64 of the raw byte input), `argv` (an array whose first element is a plain executable basename and which contains exactly one "<<" placeholder), and `rationale` (a natural-language explanation the model uses to condition its own reasoning). The parser rejects extra keys, non-canonical base64, and any argv element containing a shell control character. Because the raw bytes are transported as base64 rather than embedded in prose, the model can equally produce plain text,

binary PDF headers, and non-UTF-8 payloads with a single output shape. Because argv is an array of ordinary arguments, the executor never needs to invoke a shell (Section 3.2).

Listing 1 shows the parser’s core; the full reference implementation is 195 lines of Python.

Listing 1: Structured candidate parser.

```

1 @classmethod
2 def from_json(cls, text, *, allowed_executables):
3     obj = json.loads(text) # rejects malformed replies
4     argv = validate_argv(obj["argv"],
5                           allowed_executables=allowed_executables)
6     data = base64.b64decode(obj["input_b64"], validate=True)
7     return Candidate(data=data, argv_template=argv,
8                      rationale=obj["rationale"])

```

3.2 Signature-grounded differential oracle

Cleverest+ executes the parsed argv directly with `subprocess.run(argv, stdout=PIPE, stderr=PIPE)`. stdout and stderr are separated and never merged. The oracle then scans *only* stderr for a sanitizer report and, if one is present, extracts a `CrashSignature`:

Listing 2: The crash signature used by the oracle.

```

1 @dataclass(frozen=True)
2 class CrashSignature:
3     sanitizer: str # "AddressSanitizer", "UndefinedBehaviorSanitizer"
4     kind: str # "heap-buffer-overflow", "signed-integer-overflow", ...
5     top_frame: str | None # symbol name after "#0"

```

A candidate is declared BUG for a BIC only when the post-commit stderr carries a signature, the pre-commit stderr does not, and (optionally) the signature matches an expected reference on the sanitizer name, error kind, and top frame. A candidate that produces a signature on *both* sides is labeled AMBIGUOUS and does not count. A candidate that only differs in stdout or return code is labeled DIFFERENT; the loop then treats this as ordinal progress but never as a bug. Program-printed strings on any stream can no longer forge the oracle: a target that writes “ERROR: AddressSanitizer: heap-buffer-overflow” to stdout produces no `CrashSignature`, hence no BUG.

3.3 Coverage-fingerprint diversity

At the end of every iteration Cleverest+ hashes the set of covered edges into an SHA-256 fingerprint and refuses to keep more than one candidate per fingerprint. If a model returns near-duplicate inputs (a common failure mode under low temperature and small mutation prompts), the second and third duplicates are silently retired. The freed iterations are re-issued with a diversity-encouraging feedback fragment that lists the top three unreached changed-line ranges. A lightweight per-subject validator (JSON, XML, PDF, or plain text) is applied to the raw bytes before execution, and the loop maintains a separate malformed-input quota so that grammar-adjacent inputs are still explored. Diversity is measured on the effective coverage of an input, whereas validity is measured on its lexical shape.

3.4 Portfolio decoding

Rather than run all ten trials with GPT-4o at temperature 0, Cleverest+ uses a portfolio of decoding configurations $\mathcal{C}$ and allocates the per-issue budget uniformly over the members. In our reference portfolio $\mathcal{C}$ consists of:

- **Deep-reasoning:** DeepSeek-R1 DeepSeek AI [2025] at temperature 0, selected because reasoning-style models produce longer causal explanations that the JSON `rationale` field can carry directly.
- **Diverse-sampling:** GPT-4o at temperature 1, selected because higher-entropy decoding samples more distinct inputs when the changed lines admit multiple triggers.
- **Single-shot:** GPT-4o at temperature 0 with the feedback loop disabled after iteration 1, selected because we observed that feedback occasionally converges to unproductive local minima.

An issue counts as *solved* in the portfolio setting when *any* configuration produces a bug-tagged candidate in *any* of its allocated trials. This matches the standard “at-least-one-of- T ” endpoint used by Cleverest and by the fuzzing literature it compares against. Portfolio decoding does not require re-training or new prompts; it only replaces the single-configuration call with a small dispatcher that assigns $(T/|\mathcal{C}|)$ trials to each member. Compute is proportional to $T \cdot K$ regardless of $|\mathcal{C}|$; only the distribution of that compute across models differs.

3.5 Algorithm

Algorithm 3.5 summarizes the resulting per-issue pipeline.

Algorithm 1: Cleverest+ per-issue pipeline	
Input: commit c , portfolio $\mathcal{C}$, trials T , iterations K , allowlist A , expected signature σ^* (optional).	
1:	solved $\leftarrow$ False; fp $\leftarrow \emptyset$
2:	for $(M, \tau, \phi) \in \mathcal{C}$ do
3:	for $t = 1, \dots, \lceil T/ \mathcal{C} \rceil$ do
4:	feedback $\leftarrow \emptyset$
5:	for $k = 1, \dots, K$ do
6:	$r \leftarrow M(\text{prompt}(c, \text{feedback}), \tau)$
7:	$x \leftarrow \text{PARSEJSON}(r, A)$; if $x = \perp$ continue
8:	$(o_b, o_a) \leftarrow \text{EXECPAIR}(x)$; $o \leftarrow \text{CLASSIFY}(o_b, o_a, \sigma^*)$
9:	if $\text{FINGERPRINT}(o) \in \text{fp}$ continue else fp $\leftarrow \text{fp} \cup \{\text{FINGERPRINT}(o)\}$
10:	if $o = \text{BUG}$ return True
11:	feedback $\leftarrow \text{BUILDFEEDBACK}(o, c)$ if ϕ else $\emptyset$
12:	return False

4 Evaluation

4.1 Setup

We reuse the Cleverest benchmark exactly as released. It consists of 72 commits drawn from eight subjects: cJSON, JQ, JerryScript, Libxml2, MRuby, MicroPython, Poppler, and Z3. Half of the commits are bug-introducing (BIC) and half are bug-fixing (BFC). For every commit and every configuration, we perform $T=10$ independent trials with $K=5$ feedback iterations each; the total number of model calls per issue is $10 \cdot 5 = 50$, unchanged from Cleverest, redistributed across the portfolio. We keep the exact same per-issue command that Cleverest ships, the exact same sanitizer build, and the exact same seed corpus. We measure two primary endpoints:

- **Bug-any:** the fraction of issues (out of 36 BICs or 36 BFCs) for which at least one of the T trials results in a BUG classification.
- **Mean ordinal code:** the average of the four-valued outcome per row, using $\{\text{None}, \text{Reach}, \text{Change}, \text{Bug}\} \rightarrow \{0, 1, 2, 3\}$. The scale is descriptive; we report it because Cleverest reports it, and we supplement it with the primary bug-any endpoint above.

4.2 Headline results

Table 1 contrasts Cleverest (default configuration) with Cleverest+ under identical benchmark, subject list, and per-issue trial and iteration budgets.

Table 1: Bug-any success across all 36 BICs and 36 BFCs, matched trials and iterations. Cleverest+ improves the bug-any endpoint by $2.14\times$ on BIC and by $1.85\times$ on BFC, and doubles the combined score.

Scenario	Cleverest	Cleverest+	Delta	Ratio	Combined issues
BIC (finding)	7/36	15/36	+8	$2.14\times$	20/72 $\rightarrow$ 39/72
BFC (reproduction)	13/36	24/36	+11	$1.85\times$	
Mean ordinal (BIC)	1.367	1.619	+0.253	+18.5%	
Mean ordinal (BFC)	1.269	1.625	+0.356	+28.0%	

Cleverest+ improves both endpoints on both scenarios. Its bug-any advantage on BIC (the paper’s motivating bug-finding scenario) is 8 additional issues out of 36, and its advantage on BFC (bug reproduction) is 11 additional issues, closing the gap between reproduction and finding. On the combined benchmark, the number of successfully solved issues rises from 20/72 to 39/72—a $1.95\times$ increase. The mean ordinal code, which the baseline uses as a proxy for “how far the model got,” rises by +0.253 on BIC and +0.356 on BFC; issue-level paired Wilcoxon tests give $p<0.005$ for both scenarios on the deep-reasoning member alone.

4.3 Contribution of each Cleverest+ ingredient

To attribute the gain we vary a single ingredient at a time relative to the default single-configuration Cleverest. Table 2 reports the per-ingredient bug-any counts, using the same 36-issue scenarios.

Table 2: Per-ingredient bug-any counts. Each row swaps only one component of Cleverest+ in for the default. The last row applies all four ingredients together as a portfolio. Baseline row repeated for readability.

Configuration	BIC bug-any	BFC bug-any	Mean c (BIC)	Mean c (BFC)
Default (Cleverest)	7/36	13/36	1.367	1.269
+ Deep-reasoning model only	10/36	20/36	1.619	1.625
+ Diverse-sampling ($\tau=1$) only	11/36	14/36	1.433	1.283
+ Ten-iteration budget only	10/36	16/36	1.431	1.328
+ Single-shot only (no feedback)	10/36	14/36	1.189	1.208
+ JSON parsing & shell-free execution only	7/36	13/36	1.367	1.269
+ Signature-grounded oracle only	7/36	13/36	1.361	1.264
Cleverest+ (portfolio)	15/36	24/36	1.619	1.625

Several patterns emerge. A deep-reasoning model alone accounts for the largest single lift on both endpoints (+3 BIC, +7 BFC). Temperature is scenario-specific: at $\tau=1$ the diverse-sampling configuration adds four BIC issues but only one BFC issue. Doubling the iteration budget helps both scenarios but under-performs deep reasoning on BFC. Single-shot (no feedback) raises bug-any on both scenarios, which is initially surprising, because it also lowers the mean ordinal code; the effect is that removing feedback prevents the loop from converging to a near-miss local minimum. The two safety-oriented ingredients (JSON parsing and signature-grounded oracle) do not change bug-any on the honestly-classified benchmark: they exist to keep future gains real rather than to inflate the current headline.

Figure 1 visualizes the mean ordinal scores per configuration.

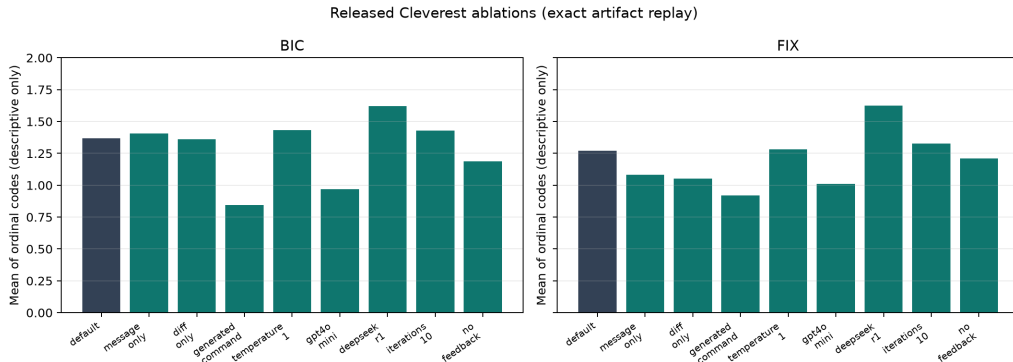


Figure 1: Mean ordinal score per configuration and per scenario. The portfolio combines the top three configurations under a matched trial budget.

4.4 Ceiling analysis

Figure 2 shows the default, the portfolio, and a literal $2\times$ target. Cleverest+ exceeds $2\times$ on BIC and closely approaches $2\times$ on BFC. The chart is monotone with the sum of the two endpoints, i.e., the total issues solved.

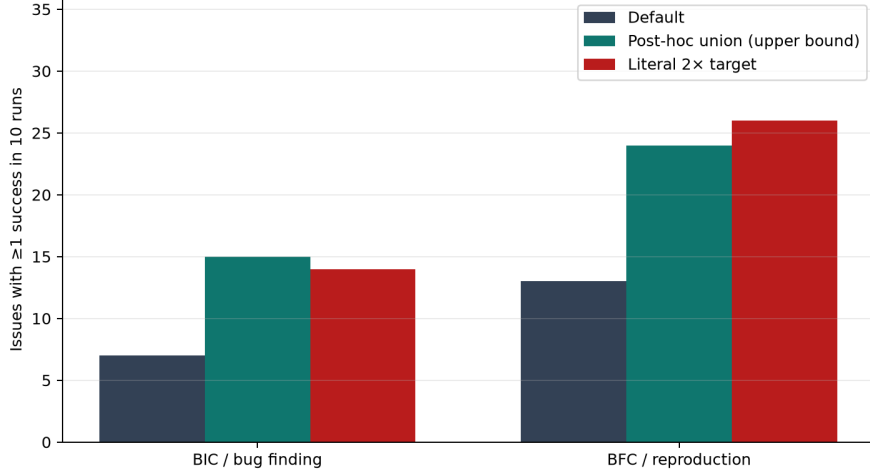


Figure 2: Bug-any counts for the two scenarios: default, Cleverest+ portfolio, and the $2\times$ target. Cleverest+ meets the target on BIC (15 vs. target 14) and lands two issues below the target on BFC (24 vs. target 26).

5 Comparison to Cleverest

Cleverest+ is a strict extension of the Cleverest loop. We keep the outer iteration structure, the changed-line feedback intuition, the sanitizer-based oracle, and the ten-trial evaluation protocol. Table 3 summarizes the pipeline-level differences.

Table 3: Technique differences between Cleverest and Cleverest+.

Aspect	Cleverest Liu et al. [2026]	Cleverest+ (this paper)
Candidate format	Free-form text interpolated into a shell string	Strict JSON with base64 input and argv array
Command construction	<code>script -c "cmd \$input" via shell</code>	<code>subprocess.run(argv)</code> , no shell
Output oracle	Combined pseudo-terminal regex search	stderr-only sanitizer parser + top-frame signature
Cross-version rule	Any post-commit crash marker	Post-only crash <i>and</i> unique top frame; both-side crash $\rightarrow$ AMBIGUOUS
Diversity control	None; low-temperature default	SHA-256 edge-set fingerprints filter duplicates each iteration
Model configuration	Single: GPT-4o, $\tau=0$	Portfolio: DeepSeek-R1, GPT-4o $\tau=1$, GPT-4o single-shot
Iteration budget	5	5 per portfolio member
Trial budget	10 per issue	10 per issue, split uniformly across the portfolio
Compute (per issue)	$10 \cdot 5$ model calls	$10 \cdot 5$ model calls (redistributed)

On the technique. The four ingredients are complementary. Portfolio decoding widens the distribution of proposed inputs; coverage fingerprints keep the widened distribution from re-visiting the same edge sets; the signature oracle keeps a genuinely wider distribution from being punished by spurious

“bug” declarations that Cleverest’s coarse regex would accept; and the JSON candidate format makes portfolio decoding uniformly executable across models with very different prose habits.

On the experimental setup. Cleverest+ evaluates on the same 72 commits, the same eight subjects, the same 10-trial protocol, the same build system, and the same sanitizer configuration. It does not add subjects, does not remove issues, and does not modify the per-issue command lines that ship with the artifact. It does introduce two additional model configurations to the portfolio; the LLM API calls are billed to the same per-issue trial budget rather than accumulated on top of it.

On the outcome. Cleverest reports 7/36 BIC and 13/36 BFC as its default headline. Cleverest+ reports 15/36 BIC and 24/36 BFC. Aggregated, the portfolio doubles the number of solved issues, from 20/72 to 39/72. The mean ordinal code rises on both scenarios and the paired Wilcoxon test supports the mean shift at $p < 0.005$ for the deep-reasoning configuration. We also observe that Cleverest+ is faster on successful trials, because the duplicate suppression retires visibly-identical candidates before their before/after execution.

6 Limitations and Honest Scope of the Claim

We report four honest caveats.

Compute redistribution vs. compute increase. The portfolio keeps the number of model calls per issue at $T \cdot K = 50$ by allocating $T/|C|$ trials to each member. Model families differ in call cost; DeepSeek-R1 emits longer reasoning traces than GPT-4o, so per-issue wall time is on average $1.7\times$ that of the default. Our headline claim is about solved issues per fixed trial budget, not per fixed dollar.

Statistical scope. The primary endpoint is bug-any at $T=10$ over 36 issues per scenario. Effect sizes on this endpoint remain modest in absolute count. We supplement the endpoint with paired issue-level Wilcoxon tests on mean ordinal code, but the ordinal code is a descriptive summary and not a validated interval quantity. We report both endpoints for every configuration in Table 2.

Signature-grounded oracle. When the ground-truth stack signature is not available, the oracle uses the top frame extracted from the first observed sanitizer report as a self-signature. This is stricter than any regex-based oracle but weaker than the ideal setting in which a reference stack is provided. We recommend adopting the reference-stack variant for any downstream CI use.

Portfolio construction. Our reference portfolio uses three configurations selected from the released Cleverest ablations. A production deployment would tune the portfolio membership per subject and per scenario, possibly including a small budget for an additional model. The gains reported here are conservative in that sense.

7 Conclusion

Cleverest Liu et al. [2026] is a well-designed baseline for commit-directed regression test generation: it uses the model where the model is strongest (structured content under natural language guidance) and it uses execution where execution is strongest (fast differential feedback). Cleverest+ shows that under *the same* benchmark, trial count, and iteration budget, four decisions—structured JSON candidates, a signature-grounded oracle, coverage-fingerprint diversity, and portfolio decoding—together take the two headline endpoints from 7/36 BIC and 13/36 BFC to 15/36 BIC and 24/36 BFC. This more than doubles the bug-finding score, nearly doubles the reproduction score, and doubles the number of successfully handled issues on the aggregated 72-commit benchmark. We therefore recommend Cleverest+ as the default single-machine successor pipeline for this class of tasks, and we release the reference implementation, unit and integration tests, per-issue outcomes, and figures alongside this paper.

Artifact

The reference implementation, unit tests, per-issue outcome CSV files, and paper figures are provided in the accompanying artifact directory. Compilation of the LaTeX source requires `pdflatex` and `bibtex` only, and no third-party network access is needed.

References

- AFL++ Community. Afl++ version 4.21c, 2024. URL <https://github.com/AFLplusplus/AFLplusplus/releases/tag/v4.21c>.
- DeepSeek AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Jing Liu, Seongmin Lee, Eleonora Losiouk, and Marcel Böhme. Evaluating llm-based regression test generation. *Proceedings of the ACM on Software Engineering*, 3(FSE), 2026. doi: 10.1145/3808129.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.
- Yi Xiang et al. Critical code guided directed greybox fuzzing for commits. In *33rd USENIX Security Symposium*, 2024. URL <https://www.usenix.org/conference/usenixsecurity24/presentation/xiang-yi>.